DAVE3606 - Project Report

Group member: Jarle Kirkeby (jakir1503@oslomet.no)

Task 1: Add database constraints

Table: `lego_brick`

This table has four columns:

- `brick_type_id` (text)
- `color_id` (integer)
- `name` (text)
- `preview_image_url` (text)

`preview_image_url` and `name` are not suitable primary keys because their values might change, and they could also not be unique.

`brick_type_id` and `color_id` we can assume will not change, but only a combination of the two would be unique. So we need to make a decision on the order. The first value in the primary key will always be indexed, but the second will only have an automatic index when the searched in combination with the first. What will be the more common pattern?

- Searching for all bricks of a certain color? Then making the first value of the composite key `color_id` would make sense, since it would be sorted by this value first.
- Searching for all the colors for a certain brick? Then we would want the first value to be `brick_type_id`.

I decided to make the composite key `brick_type_id` - `color_id` because I think the second pattern will be more common. Users will look up sets, then go to the individual bricks and look at color variations for that brick. Finding all bricks of a certain color is a more niche request, but an index can be made for the color column if necessary.

To add the primary key I connected to the database, and used the command:

```
ALTER TABLE lego_brick ADD PRIMARY KEY (brick_type_id, color_id);
```

Table: `lego_inventory`

This table also has four columns:

- `set_id` (text)
- `brick_type_id` (text)
- `color_id` (integer)

- `count` (integer)

The `count` value is not an identifier as many combinations of sets, bricks and color can have the same count. So here it is clear that the primary key should be some combination of `set_id`, `brick_type_id` and `color_id`. I have already decided how to best combine brick and color, so where should we put the set?

- Sort by set first? Then we will have the table sorted by set -> brick -> color. Which is suitable if the primary use case is to retrieve sets, then their bricks and color.
- Sort by brick/color first? Then we would expect users to look for specific bricks and colors, and then wanting to find all sets that use them.

Sorting by set seems to support the most common use case: "finding a set, and then viewing the inventory for that set". The other option is more niche, but can be supported by manually adding indexes.

```
ALTER TABLE lego_inventory ADD PRIMARY KEY (set_id, brick_type_id, color_id);
```

When I tried to add this constraint, I got an error:

```
ERROR:  could not create unique index "lego_inventory_pkey"
DETAIL:  Key (set_id, brick_type_id, color_id)=(10218-1, 48729b, 11) is
duplicated.
```

I investigated the specific IDs and found that the set had multiple counts for the same brick:

```
lego-db=# SELECT * FROM lego_inventory WHERE set_id = '10218-1' AND brick_type_id
= '48729b';
 set_id  | brick_type_id | color_id | count
---------+---------------+----------+-------
 10218-1 | 48729b        |       11 |     2
 10218-1 | 48729b        |       86 |     2
 10218-1 | 48729b        |       11 |     1
 10218-1 | 48729b        |       86 |     1
(4 rows)
```

To solve this we need to either change the data (combine the lines into single lines with 3 count for each). If we don't we must introduce a surrogate key since there are no valid natural keys for the table.

I chose to combine the rows since I don't consider it necessary to store multiple counts for a combination of set/brick/color. If this is necessary a bag (or brick group) id should be added to group bricks within a set together.

```
DELETE FROM lego_inventory WHERE set_id = '10218-1' AND brick_type_id = '48729b';
INSERT INTO lego_inventory VALUES ('10218-1', '48729b', 11, 3), ('10218-1',
'48729b', 86, 3);

# Etter endring:
 set_id  | brick_type_id | color_id | count
---------+---------------+----------+-------
 10218-1 | 48729b        |       11 |     3
 10218-1 | 48729b        |       86 |     3
(2 rows)
```

After combining I found that there were more rows in the table where the same issue prevented creating the primary key. So I used this transaction to combine all of them:

```
BEGIN;
CREATE TEMP TABLE temp_lego_inventory AS (SELECT set_id, brick_type_id, color_id,
SUM(count) as count FROM lego_inventory GROUP BY set_id, brick_type_id, color_id);
# Feedback from server: SELECT 1199929

DELETE FROM lego_inventory;
# Feedback form server: DELETE 1301257

INSERT INTO lego_inventory (set_id, brick_type_id, color_id, count) SELECT set_id,
brick_type_id, color_id, count FROM temp_lego_inventory;
# Feedback from server: INSERT 0 1199929

DROP TABLE temp_lego_inventory;
COMMIT;
```

Based on the feedback we can see the `1301257 - 1199929 = 101318` lines where combined. Which is around 7% if the table, so this is a pretty big decision to make - maybe the separation of these values had some meaning in the original data which has now been removed (but that link was already severed when the data was changed into its current state for the assignment before the lines were combined).

I was able to add the primary key after making these changes.

Table: `lego_set`

This table has an `id` column that already fulfills the requirements for a primary key, so I added that as the primary key:

```
ALTER TABLE lego_set ADD PRIMARY KEY (id);
```

Task 2: Design indexes for flexible queries

Query: Which LEGO sets contain a specific brick type regardless of color?

Testing with the first, middle (OFFSET 500000) and last `brick_type_id` in the `lego_inventory` table: *29bc01*, *3062*, *sw0517a*

```
SELECT set_id FROM lego_inventory WHERE brick_type_id = '29bc01';
# (32 rows)
# Time: 131.496 ms

SELECT set_id FROM lego_inventory WHERE brick_type_id = '3062';
# (7640 rows)
# Time: 123.281 ms

SELECT set_id FROM lego_inventory WHERE brick_type_id = 'sw0517a';
# (1 row)
# Time: 130.074 ms
```

The query will have to go visit each `set_id` since that's the first value in the composite key. But within each set the rows are sorted on `brick_type_id`, so the database should be able to quickly check if the brick is in that set or not (binary search). If the composite key had `brick_type_id` as its first value this search would be optimized for (table would be sorted by bricktype).

I added an index for `brick_type_id`:

```
CREATE INDEX ON lego_inventory(brick_type_id);
```

And re-tested the queries:

```
SELECT set_id FROM lego_inventory WHERE brick_type_id = '29bc01';
# (32 rows)
# Time: 4.305 ms

SELECT set_id FROM lego_inventory WHERE brick_type_id = '3062';
# (7640 rows)
# Time: 88.073 ms

SELECT set_id FROM lego_inventory WHERE brick_type_id = 'sw0517a';
# (1 row)
# Time: 0.440 ms
```

We see quite large improvements for the brick types with few results as the database is quickly able to find the bricks, and then get the sets for each. For the second result the improvement was OK, but would have expected more. Maybe if I indexed on both `brick_type_id` and `set_id` I would see more improvement:

```
CREATE INDEX ON lego_inventory(brick_type_id, set_id);

SELECT set_id FROM lego_inventory WHERE brick_type_id = '29bc01';
# (32 rows)
# Time: 0.417 ms

SELECT set_id FROM lego_inventory WHERE brick_type_id = '3062';
# (7640 rows)
# Time: 1.930 ms

SELECT set_id FROM lego_inventory WHERE brick_type_id = 'sw0517a';
# (1 row)
# Time: 0.440 ms
```

This improved the queries (especially the second) because now the database also knows the sets are ordered in the index and can quickly get all the sets for a brick type in order.

Query: Which LEGO sets contain bricks of a specific color, regardless of type?

I used a query to find which `color_id`s were most used, least used and median used:

```
SELECT color_id, COUNT(set_id) AS count FROM lego_inventory GROUP BY color_id
ORDER BY count DESC;
# (212 rows)
# Most used: 11 (205682 times)
# Least used: 132 (1 time) -- others also had 1 occurence

# Using offset to find the 106th element
SELECT color_id, COUNT(set_id) AS count FROM lego_inventory GROUP BY color_id
ORDER BY count DESC LIMIT 1 OFFSET 105;
# Median used: 102 (68 times)
```

So we will use these `color_id`s to test the query before and after adding the index: *11*, *102*, *132*.

```
SELECT set_id FROM lego_inventory WHERE color_id = 11;
# Time: 424.356 ms

SELECT set_id FROM lego_inventory WHERE color_id = 102;
# Time: 135.844 ms

SELECT set_id FROM lego_inventory WHERE color_id = 132;
# Time: 139.596 ms
```

Now we add an index for `(color_id, set_id)` to speed up the query and re-test:

```
CREATE INDEX ON lego_inventory(color_id, set_id);

SELECT set_id FROM lego_inventory WHERE color_id = 11;
# Time: 35.528 ms

SELECT set_id FROM lego_inventory WHERE color_id = 102;
# Time: 0.332 ms

SELECT set_id FROM lego_inventory WHERE color_id = 132;
# Time: 0.409 ms
```

Again we see a large improvement, and for colors used in few sets the database can resolve the query almost instantly.

Task 3: Algorithmic complexity improvements

First I tested loading all the sets in the browser, the console tells me the time it took:

```
127.0.0.1 - - [15/Mar/2026 11:38:26] "GET / HTTP/1.1" 200 -
Time to render all sets: 1.0528725479998684
```

Looking at the main loop of `/sets` endpoint we can see that this is the classic example of the **painter's algorithm** where a string is expanded by copying the entire string so far and the new addition.

```
for row in cur.fetchall():
    html_safe_id = html.escape(row[0])
    html_safe_name = html.escape(row[1])
    existing_rows = rows
    rows = existing_rows + f'<tr><td><a href="/set?id={html_safe_id}">
{html_safe_id}</a></td><td>{html_safe_name}</td></tr>\n'
```

Each time you add a row you need to re-do the work *k* (writing a line) one more time, and for the last addition you do *k (n-1)* times which makes the total complexity $\Theta(kn^2)$. This makes this algorithm which should have a complexity of *n* iterating through a list have a complexity of $\Theta(n^2)$.

To solve the problem I keep all the rows in a list, and then join them all at once at the end. Which means we just iterate through a list of *n* elements twice. Complexity: $\Theta(2n) = \Theta(n)$.

```
template = open("templates/sets.html").read()
rows = [] # List to hold the rows to avoid painter's algorithm

start_time = perf_counter()
conn = psycopg.connect(**DB_CONFIG)
try:
    with conn.cursor() as cur:
        cur.execute("select id, name from lego_set order by id")
        for row in cur.fetchall():
            html_safe_id = html.escape(row[0])
            html_safe_name = html.escape(row[1])
            rows.append(f'<tr><td><a href="/set?id={html_safe_id}">{html_safe_id}
</a></td><td>{html_safe_name}</td></tr>\n')
    print(f"Time to render all sets: {perf_counter() - start_time}")
finally:
    conn.close()

result = "".join(rows) # Combine the strings efficiently, all at once
page_html = template.replace("{ROWS}", result)
return Response(page_html, content_type="text/html")
```

With these changes running the request again to see how long it took:

```
127.0.0.1 - - [15/Mar/2026 12:10:07] "GET /sets HTTP/1.1" 200 -
Time to render all sets: 0.057146040999214165
```

Task 4: Encoding, compression, and file handle leaks

First I added the query parameter for encoding with a default value as `'UTF-8'` and checked that the value was one of the valid encodings:

```
@app.route("/sets")
def sets():
    valid_encodings = ['UTF-16-LE', 'UTF-16-BE', 'UTF-32-LE', 'UTF-32-BE', 'UTF-
8']

    requested_encoding = request.args.get('encoding', default='UTF-8')

    encoding = requested_encoding if requested_encoding in valid_encodings else
'UTF-8'
```

Then I made sure to actually use the encoding on the HTML, by:

1. Changing the template (`sets.html`) to take in an encoding:

   ```
   <meta charset="{ENCODING}">
   ```

2. Applying a second replace when creating the HTML (in the `/sets` endpoint):

   ```
   page_html = template.replace("{ROWS}", result).replace("{ENCODING}", encoding)
   ```

3. Encoding and passing the bytes to flask response:

   ```
   # Manually encode the bytes
   response_bytes = page_html.encode(encoding)

   return Response(response_bytes, content_type=f"text/html; charset={encoding}")
   ```

This worked in that I got the responses to send, and got different sizes for different encodings. But only UTF-8 encoded HTML would display correctly (the others just displayed the raw HTML with tags). After some troubleshooting with AI, I found that the issue was caused by the browser (Firefox), which would only show UTF-8 and UTF-16 (without -LE and -BE). I added UTF-16 to the valid encodings, and it rendered properly. Since I confirmed the problem was out of my control, I just continued with the assignment without fixing this issue.

I tested different encodings to see the difference in sizes:

Encoding	Size
UTF-8	2MB
UTF-16-LE	4MB
UTF-16-BE	4MB
UTF-32-LE	8MB
UTF-32-BE	8MB

Then I added compression using the `gzip` library, and set the `Content-Encoding` header field:

```
response_bytes = page_html.encode(encoding)
compressed_bytes = gzip.compress(response_bytes)

return Response(
    compressed_bytes,
    headers = {
        'Content-Type': f'text/html; charset={encoding}',
        'Content-Encoding': 'gzip'
    })
```

Which gave these updated size when tested:

Encoding	Size	Size with `gzip`
UTF-8	2MB	335 kB
UTF-16-LE	4MB	390 kB
UTF-16-BE	4MB	390 kB
UTF-32-LE	8MB	450 kB
UTF-32-BE	8MB	450 kB

Closing file handles

When the templates are opened we don't use `try-catch-finally` or `with` to ensure that the file handles are closed:

```
template = open("templates/index.html").read()
```

I replaced this with the safer pattern that reads into a string and then closes the file handle:

```
with open("templates/index.html") as f:
    template = f.read()
```

Task 5: File formats

Endpoint: `/api/set`

I took inspiration from the database connection in the `/sets` endpoint to create the database queries for a new `/api/set` endpoint:

- First I query the `lego_set` table to retrieve the set information using a prepared statement as the `id` is a string (vulnerable to SQL Injection):

  ```
  cur.execute(
              "SELECT year, name, category, preview_image_url FROM lego_set WHERE
  id = %s;",
              [set_id],
              prepare=True)
  ```

- Then I store the use `html.escape(value)` to clean the strings (the integers are safe):

  ```
  result = cur.fetchone()
          html_safe_year = result[0] # No need for html.escape() on integers
          html_safe_name = html.escape(result[1])
          html_safe_category = html.escape(result[2])
          html_safe_preview_image_url = html.escape(result[3])
  ```

- To retrieve the information for each inventory item, I use a JOIN-statement to combine information in the `lego_inventory` and `lego_brick` tables:

  ```
  cur.execute(
              "SELECT set_id, i.brick_type_id, i.color_id, count, name,
  preview_image_url " \
              "FROM (lego_inventory AS i INNER JOIN lego_brick AS b ON
  i.brick_type_id = b.brick_type_id AND i.color_id = b.color_id) " \
              "WHERE set_id = %s;",
              [set_id],
              prepare=True)
  ```

- Since the result can have many lines, I iterate through them and store the values in a dictionary which I then add to a list (using `html.escape(value)` on the strings):

```
for item in cur.fetchall():
            html_safe_item = {}
            html_safe_item["brick_type_id"] = html.escape(item[1])
            html_safe_item["color_id"] = item[2]
            html_safe_item["count"] = item[3]
            html_safe_item["name"] = html.escape(item[4])
            html_safe_item["preview_image_url"] = html.escape(item[5])

            # Add the item to the inventory
            html_safe_inventory.append(html_safe_item)
```

- Lastly, I compose the result into a dictionary:

```
result = {
    "set_id": set_id,
    "year": html_safe_year,
    "name": html_safe_name,
    "category": html_safe_category,
    "preview_image_url": html_safe_preview_image_url,
    "inventory": html_safe_inventory
}
```

Example of the resulting JSON output:

```
{
    "set_id": "0011-2",
    "year": 1982,
    "name": "LEGOLAND Mini-Figures",
    "category": "Catalog: Sets: Town: Classic Town: Supplemental",
    "preview_image_url": "https://img.bricklink.com/ItemImage/ST/0/0011-2.t1.png",
    "inventory": [
        {
            "brick_type_id": "cop001",
            "color_id": 0,
            "count": 1,
            "name": "Police - Suit with 4 Buttons, Black Legs, White Hat",
            "preview_image_url": "https://img.bricklink.com/M/cop001.jpg"
        },
        {
            "brick_type_id": "pln024",
            "color_id": 0,
            "count": 1,
            "name": "Plain Blue Torso with Blue Arms, Red Legs, Black Pigtails
Hair",
            "preview_image_url": "https://img.bricklink.com/M/pln024.jpg"
        },
        {
            "brick_type_id": "pln026",
            "color_id": 0,
            "count": 1,
            "name": "Plain Red Torso with Red Arms, Blue Legs, Red Construction
Helmet",
            "preview_image_url": "https://img.bricklink.com/M/pln026.jpg"
        }
    ]
}
```


Designing my own binary format

First I created a new endpoint `/api/set/bin`, and copied over a lot of the code from the `api/set` which retrieves the set information and inventory and stores it in a dictionary.

```
@app.route("api/set/bin")
def apiSetBin():
    ...
```

I removed the `html.escape()` calls, since we want the binary to show exactly what is in the database.

The next task is to encode this into a binary format in a way that enables it to be decoded into the same dictionary. Looking at the example from the last part of the task we can determine the structure of the dictionary.

- First there are set properties that can be packed directly into binary and unpacked.
- Then there is an inventory that contain brick information, but since this is just a list and it's at the end of the structure we can safely just read any values after the set properties as bricks - and append them to the list.
- Some values can be null, so for these we include a single byte to indicate if the value is present or not where 0 represents a null value.

My format looked like this (using big-endian endianness):

Set information

(?) indicates the value is not present if null.

Format Character	Bytes	Function
H	2	Length of `set_id`
c	n	`set_id`
B	1	`year` is null?
H	2	`year` (?)
H	2	Length of `name`
c	n	`name`
B	1	`category` is null?
H	2	Length of `category`
c	n	`category` (?)
B	1	`preview_image_url` is null?
H	2	Length of `preview_image_url`
c	n	`preview_image_url` (?)

Inventory lines information

This may be an empty list or contain several values.

Format Character	Bytes	Function
H	2	Length of `brick_type_id`
c	n	`brick_type_id`
B	1	`color_id`
H	2	`count`
H	2	Length of `name`
c	n	`name`
B	1	`preview_image_url` is null?
H	2	Length of `preview_image_url`
c	n	`preview_image_url` (?)

Reading the binary format

I created the script `print_lego_binary` that reads the data from a binary file created by the `/api/set/bin` endpoint. It reads the data into a Python dictionary and then prints it in the common JSON format (with indentations). To run it use `python3 print_lego_binary.py [binary-filename]`

Example output:

```
$ python3 print_lego_binary.py bin
{
    "set_id": "0016-1",
    "year": 1982,
    "name": "Knights",
    "category": "Catalog: Sets: Castle: Classic Castle",
    "preview_image_url": "https://img.bricklink.com/ItemImage/ST/0/0016-1.t1.png",
    "inventory": [
        {
            "brick_type_id": "3847a",
            "color_id": 9,
            "count": 3,
            "name": "Light Gray Minifigure, Weapon Sword, Shortsword - Polished
Rigid ABS",
            "preview_image_url": "https://img.bricklink.com/ItemImage/ST/0/0016-
1.t1.png"
        },
        {
            "brick_type_id": "cas229",
            "color_id": 0,
            "count": 1,
            "name": "Classic - Knights Tournament Knight Black, Red Legs with
Black Hips, Light Gray Neck-Protector",
            "preview_image_url": "https://img.bricklink.com/ItemImage/ST/0/0016-
1.t1.png"
        },
        {
            "brick_type_id": "cas230",
            "color_id": 0,
            "count": 1,
            "name": "Classic - Knights Tournament Knight Red, Black Legs with Red
Hips",
            "preview_image_url": "https://img.bricklink.com/ItemImage/ST/0/0016-
1.t1.png"
        },
        {
            "brick_type_id": "cas233",
            "color_id": 0,
            "count": 1,
            "name": "Classic - Knight, Shield Red/Gray, Light Gray Legs with Red
Hips, Light Gray Neck-Protector",
            "preview_image_url": "https://img.bricklink.com/ItemImage/ST/0/0016-
```

```
1.t1.png"
        }
    ]
}
```

Task 6: Frontend and caching

Fixing the frontend

First I updated the `set.html` file to correctly display the data from the JSON sent from the API:

```
fetch(`/api/set?id=${setId}`)
    .then(function(setData) { return setData.json(); })
    .then(function(setJson) {
        document.getElementById("setId").textContent = setJson.set_id;
        document.getElementById("setName").textContent = setJson.name;

        const inventoryTable = document.getElementById("inventory");

        setJson.inventory.forEach(item => {
            const inventoryRow = document.createElement("tr");

            inventoryRow.innerHTML = `
                <td><img src="${item.preview_image_url}" width="50"></td>
                <td>${item.brick_type_id}</td>
                <td>${item.color_id}</td>
                <td>${item.count}</td>
            `;

            inventoryTable.appendChild(inventoryRow);
        });
    })
    .catch(function(err) {
        document.getElementById("error").textContent = "Failed to load data: " +
err;
    });
```

Implementing a cache

To implement the cache I used an `OrderedDict` which makes it possible to keep track of which object was Least Recently Used (LRU) by moving objects to the end when used. To implement the limit of 100, I made it so the `put` function evicts the first (LRU) element in the dictionary:

```python
from collections import OrderedDict

class SimpleCache:
    def __init__(self, capacity=100):
        self.cache = OrderedDict()
        self.capacity = capacity

    def get(self, key):
        if key not in self.cache:
            return None

        # Move to end to make it "Least Recently Used" (LRU)
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key, value):
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value

        # If we exceed capacity, remove the first (oldest) item
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)
```

In the `/set` endpoint I use the `set_id` to check if it has been cached, and if so I return the cached result. I need to take into account that the endpoint may be called without a `set_id`, in that case I cache a default page.

```
@app.route("/set")
def legoSet():
    set_id = request.args.get("id")

    if set_id is None:
        set_id = "Default"

    cache_result = cache.get(set_id)
    if cache_result:
        return cache_result

    # Not in cache:
    with open("templates/set.html") as f:
        template = f.read()
    response = Response(template)
    cache.put(set_id, response) # Add to cache
    return response
```

Testing the cache

I then added timers with `perf_counter()` to see the difference between cache misses and hits (I converted it to ms to better understand the difference):

```
Cache miss: 0.13070300155959558 ms
Cache hit: 0.007753998943371698 ms
```

Which mean it was ~17x faster to retrieve it from cache.

Adding `Cache-Control` header

To make the browser cache the result for 60 seconds I added an `max-age=60` to the `Cache-Control` field in the header:

```
return Response(
        compressed_bytes,
        headers = {
            'Content-Type': f'text/html; charset={encoding}',
            'Content-Encoding': 'gzip',
            'Cache-Control': 'max-age=60'
        })
```

It worked, but I learned that the refresh icon (and F5) do a "hard refresh" which ignores the cache, but clicking the URL bar and `Enter` to reload used the cached result.

Task 7: Testing and dependency injection

Creating the Database class

First I defined the `Database` class in `database.py`:

```
class Database:
    def __init__(self):
        self.conn = psycopg.connect(**DB_CONFIG)
        self.cur = self.conn.cursor()

    def execute_and_fetch_all(self, query):
        self.cur.execute(query)
        return self.cur.fetchall()

    def close(self):
        self.cur.close()
        self.conn.close()
```

Then I updated the endpoints. By changing this:

```
conn = psycopg.connect(**DB_CONFIG)
try:
    with conn.cursor() as cur:
        cur.execute("select id, name from lego_set order by id")
        for row in cur.fetchall():
            html_safe_id = html.escape(row[0])
            html_safe_name = html.escape(row[1])
            rows.append(f'<tr><td><a href="/set?id={html_safe_id}">{html_safe_id}
</a></td><td>{html_safe_name}</td></tr>\n')
    print(f"Time to render all sets: {(perf_counter() - start_time)}")
finally:
    conn.close()
```

Into this (and equivalent for the other endpoints):

```
db = Database()
try:
    db_result = db.execute_and_fetch_all("select id, name from lego_set order by
id")

    for row in db_result:
        html_safe_id = html.escape(row[0])
        html_safe_name = html.escape(row[1])
        rows.append(f'<tr><td><a href="/set?id={html_safe_id}">{html_safe_id}</a>
</td><td>{html_safe_name}</td></tr>\n')

    print(f"Time to render all sets: {(perf_counter() - start_time)}")
finally:
    db.close()
```

Some of the endpoints used parameterized queries, in order to pass this we need to update `execute_and_fetch_all` function:

```
def execute_and_fetch_all(self, query, vars=None, **kwargs):
        self.cur.execute(query, vars, **kwargs)
        return self.cur.fetchall()
```

Now we can send pass along parameters for the query, and other parameters for the `execute` function which are stored in the `**kwargs`:

```
db_result_set = db.execute_and_fetch_all("SELECT year, name, category,
preview_image_url FROM lego_set WHERE id = %s;",
            [set_id],
            prepare=True)
```

Testing

To use the Dependency Injection pattern we need to move the actions done towards the database into separate functions that take a database as a parameter. We essentially move the logic inside the `try` into a separate function:

```
def get_sets_list_html(db):
    rows = []

    start_time = perf_counter()
    db_result = db.execute_and_fetch_all("select id, name from lego_set order by
id")

    for row in db_result:
        html_safe_id = html.escape(row[0])
        html_safe_name = html.escape(row[1])
        rows.append(f'<tr><td><a href="/set?id={html_safe_id}">{html_safe_id}</a>
</td><td>{html_safe_name}</td></tr>\n')

    print(f"Time to render all sets: {(perf_counter() - start_time)}")
    return rows
```

And in the endpoint:

```
db = Database()
try:
    rows = get_sets_list_html(db)
finally:
    db.close()
```

We can then create a `MockDatabase` that returns what we expect from the database, and we can use this to test if the function returns the expected output. We also check if the function performs the expected query:

```
class MockDatabase:
    def __init__(self, db_result, expected_query):
        self.db_result = db_result
        self.expected_query = expected_query

    def execute_and_fetch_all(self, query):
        if (query != self.expected_query):
            print("MockDatabase: Unexpected query!")
            return ()

        return self.db_result

    def close():
        return
```

For the function `get_sets_list_html` the database should return some a tuple containing some rows (also tuples) with (`id`, `name`). Then the function should return a list of html rows containing the info:

```
def test_get_sets_list_html():
    # Arrange
    # Setting up the mock database with return values
    set_id1 = "123-ab"
    name1 = "Lego set 1"
    set_id2 = "456-cd"
    name2 = "Lego set 2"

    db = MockDatabase(((set_id1, name1), (set_id2, name2)))

    # Act
    # Performing the function call
    result = get_sets_list_html(db)

    # Assert
    # Check that the result is matching what we expect
    expected_result = [
        f'<tr><td><a href="/set?id={set_id1}">{set_id1}</a></td><td>{name1}</td>
</tr>\n',
        f'<tr><td><a href="/set?id={set_id2}">{set_id2}</a></td><td>{name2}</td>
</tr>\n',
    ]

    if result == expected_result:
        print("test_get_sets_list_html: Test passed!")
    else:
        print("test_get_sets_list_html: Test failed! Unexpected result.")
```

I created similar tests for the other functions that use a database in the file `testing.py`. To run the tests just run that file with python. Since some of my functions made multiple queries, I changed the `MockDatabase` to accept a list of `expected_queries` and `db_results` instead.